

Empirical Correlation Maximization

Alexander Faust

I. Introduction & Motivation

This project builds on the findings from Vidales et. al. and build on their *Unbiased Deep Solvers for Linear Parametric PDE's* code base. High-dimensional partial differential equations arise throughout computational finance, most notably in the pricing of multi-asset options. Classical numerical schemes such as finite differences suffer from the curse of dimensionality, making them impractical beyond a handful of dimensions. Neural networks trained by stochastic gradient descent can approximate solutions to high-dimensional PDE's without the exponential cost of grid-based methods.

The author's work is about developing deep learning algorithms for pricing European options. Their central idea is to combine a neural network with Monte Carlo simulation through the Martingale Representation Theorem. An unbiased control-variate estimator is then produced regardless of network quality. A poor network only reduces the variance benefit, not the correctness of the price.

This paper proposes several learning algorithms but the accompanying code base implements only a subset of them. In particular Algorithm 5 in Appendix B is left out (the empirical correlation-maximization solver). The goal of this project is to implement Algorithm 5 within the existing code base and evaluate it against the algorithms already provided, under a common training budget and on a problem with a known closed-form solution.

II. Background: Author's Framework

The pricing problem considered is a linear parabolic PDE whose solution $v(t, x)$ represents an option price. By the Feynman-Kac theorem, this solution yields a probabilistic representation as a discounted expected payoff,

$$v(t, x) = E[D(t, T)g(X_T) | X_t = x]$$

where X_T is an asset-price process solving a SDE, $g(X_T)$ is the payoff, and $D(t, T)$ is a stochastic discount factor. This representation is what allows Monte Carlo methods to price the option: simulate many paths, average the discounted payoff. The problem is slow convergence; halving the Monte Carlo error requires increasing the number of samples by 4x.

By introducing control variates derived from the PDE itself, one can reduce the variance of the option price. Applying Itô's formula to the discounted solution along a simulated path causes all drift terms to cancel by the PDE, leaving the stochastic integral:

$$v(t, x) = D(t, T)g(X_T) - \int_t^T D(t, s) \partial_x v \sigma dW_s$$

The stochastic integral on the right is a martingale with expectation zero, yet it is perfectly correlated with the discounted payoff. Subtracting it leaves the mean unchanged while cancelling the variance; in principle producing a zero-variance estimator. This integral is the ideal control variate. In practice this integral cannot be used directly since it relies on the unknown gradient $\partial_x v$. The algorithms in the paper approximate this gradient using a neural network. Once trained, the network is plugged into a discretized version of the control variate Monte Carlo estimator as seen in equation (9). Lastly, since the integral is zero-mean regardless of the network, the estimator stays unbiased – the quality of the network only affects how much variance is removed.

The algorithms presented in the paper train the gradient network in different ways. Algorithm 2 (projection solver) learns the price v using least-squares regression and recovers the gradient by automatic differentiation. Algorithm 3 (BSDE solver) learns the gradient directly from the martingale representation. As for the other algorithms shown off in this paper, they are of no interest for this exercise as they are not being compared against Algorithm 5 – discussed in the next section.

III. Derivation of Correlation-Maximization Loss

Algorithm 5 trains the gradient network by maximizing the correlation between the discounted payoff and the control variate, rather than minimizing a pointwise approximation error. The derivation proceeds from the variance of the control-variate estimator. The unbiased estimator from equation (9) in the paper has the form

$$V^{CV} = \Xi_T - \lambda M \quad | \quad \Xi_T = D(0, T)g(X_T)$$

Where Ξ_T is the discounted payoff, M is the discrete stochastic integral control variate built from the gradient network, and λ is a scaling parameter. Because M is a martingale with $E[M] = 0$ the estimator is unbiased for any choice of λ .

Expanding the variance of V^{CV} :

$$Var(V^{CV}) = Var(\Xi_T) - 2\lambda Cov(\Xi_T, M) + \lambda^2 Var(M)$$

Minimizing over λ gives the optimal multiplier (already in the code base):

$$\lambda^* = \frac{Cov(\Xi_T, M)}{Var(M)}$$

Substituting λ^* back into the variance and factoring out $(Var(\Xi_T))$ yields an expression in terms of the Pearson correlation coefficient ρ between Ξ_T and M :

$$\frac{Var(V^{CV})|_{\lambda=\lambda^*}}{Var(\Xi_T)} = 1 - \rho_{\Xi_T, M^\theta}^2$$

The variance $Var(\Xi_T)$ of the naïve estimator is fixed by the problem and cannot be changed by training. The only term the gradient network influences is ρ^2 , through its effect on M . Minimizing the estimator variance is equivalent to maximizing the squared correlation, giving the training objective:

$$\theta^* = \arg \min_{\theta} \left(1 - \rho_{\Xi_T, M^\theta}^2 \right)$$

Conceptually this differs from the BSDE solver, Algorithm 3, which minimizes a pointwise residual and requires the network to match the gradient $\partial_x v$ in both shape and magnitude. The correlation objective requires only a control variate correlated with the payoff to be learned. Essentially learning the shape/direction of the payoff while handling scaling with λ^* . The learning target is easier with correlation-maximization leading to improved results.

The loss is computed on each minibatch as the empirical squared correlation:

$$\mathcal{L}(\theta) = 1 - \frac{\widehat{Cov}(\Xi_T, M^\theta)^2}{\widehat{Var}(\Xi_T)\widehat{Var}(M^\theta)}$$

Note, the hatted quantities are sample estimates over the batch. The loss is bounded in $[0,1]$ i.e. it equals zero when the control variate is perfectly correlated with the payoff and approaches one when the network output is uncorrelated with it.

IV. Implementation

My version of algorithm 5 was implemented by adding a new training method, `corr_max` to the `FBSDE` class in `lib/bsde_risk_neutral_measure.py`. The interface of algorithm 5 mirrors the existing `bsdeint` and `l2_proj` methods implemented by the authors making it a simple drop in to their training loop.

This method simulates a batch of asset paths and Brownian increments (with gradient tracking enabled, so the network can be updated), evaluates the gradient network along those paths, computes the discrete stochastic integral M using equation (9), and computes the discounted payoff Ξ_T . The loss is as described in Section 3 as $\mathcal{L}(\theta)$.

```
# Loss = 1 - squared empirical Pearson correlation(mc, stoch_int).
mc_centered = mc - mc.mean()
m_centered = stoch_int - stoch_int.mean()
cov = (mc_centered * m_centered).mean()
var_xi = mc_centered.pow(2).mean()
var_m = m_centered.pow(2).mean()
loss = 1 - (cov ** 2) / (var_xi * var_m + 1e-8)

return loss, Y, payoff
```

The loss depends only on the gradient network `self.dfdx`, not the price network, since only the gradient is needed to build the control variate. All ‘corr_max’ models are passed to the existing ‘unbiased_price’ bsde branch which reads `self.dfdx` directly. The final change made was to add a `corr_max` option as a third –method choice in `pde_BlackScholes_exchange.py`.

V. Experimental Results

The three training methods were compared on the two-asset exchange option, chosen because it has a closed-form price given by the Margrabe formula, providing an exact benchmark. Each method was trained for 2,000 iterations under identical settings, repeated over three random seeds. After training, each model’s control-variate estimator was evaluated by Monte Carlo, and its variance was compared against the plain Monte Carlo estimator. The results are summarized below.

Method	Variance Reduction Factor	Bias from Margrabe
bsde	91.5 ± 7.0	-4.3×10^{-5}
l2_proj	15.7 ± 6.8	-4.2×10^{-4}
corr_max	99.3 ± 10.8	-1.6×10^{-5}

VI. Discussion and Conclusion

The results support two conclusions. First, all three methods produce unbiased price estimates: every control-variate estimator agreed with the closed-form Margrabe price. The Margrabe price (0.11924) is the exact, true price of the exchange option. Furthermore, the Margrabe formula is a closed-form solution which can be used to determine the accuracy of each method’s predicted price. Each of the three methods show very small order biases which can be contributed to Monte

Carlo noise. These results illustrate the key property of each method – the neural network influences only the variance of the estimator, never its correctness.

The second conclusion is that at an equal training budget, the correlation-maximization solver performed on par with the BSDE solver, with a variance reduction factor of 99.3 against 91.5. The difference between `corr_max` and `bsde` is within roughly one standard deviation, so the two are best regarded as comparable, with `corr_max` being slightly ahead in effectiveness.